

Assignment 1: BBS

Software Engineering - UATX - Spring 2026

Due	One week after assigned
Estimated time	8-12 hours (with coding agent)
Prereqs	Lectures 2.1 (Files & Data Formats) and 2.2 (Databases & SQL)
Submit	Branch + PR on the class repo

Background

Before the World Wide Web, there were **bulletin board systems** (BBSes). Starting in the late 1970s, hobbyists ran BBS software on home computers connected to phone lines. You'd dial in with your modem, and you'd land on a text-based interface where you could read messages, post replies, and chat with other users. All through a terminal.

BBSes were the internet before the internet. They had forums, file sharing, multiplayer games ("door games"), and thriving communities, all over a single phone line, one user at a time. At their peak in the mid-1990s there were an estimated 150,000 BBSes in North America alone.

In this assignment, you'll build one. Yours will be simpler than the originals (no modem required), but the idea is the same: a terminal-based program where users can post messages, read what others have written, and interact through text.

You'll build it twice. First using JSON files for storage, then upgrading to a real SQLite database. Then you'll write a migration script that bridges the two, and see firsthand why databases exist.

Part A: JSON File Storage

Build a command-line Python program called `bbs.py` that stores all data in a JSON file called `bbs.json`.

Required commands

```
python bbs.py post <username> <message>      # Post a message
python bbs.py read                             # Read all messages
python bbs.py users                           # List all users who have
posted
python bbs.py search <keyword>                # Search posts by keyword
```

Expected behavior

```
$ python bbs.py post alice "Hello, is anyone out there?"
Posted.

$ python bbs.py post bob "Hey Alice! Welcome to the board."
Posted.

$ python bbs.py read
[2026-03-24 14:01] alice: Hello, is anyone out there?
[2026-03-24 14:03] bob: Hey Alice! Welcome to the board.

$ python bbs.py users
alice
bob

$ python bbs.py search "Hello"
[2026-03-24 14:01] alice: Hello, is anyone out there?

$ cat bbs.json
[
  {
    "username": "alice",
    "message": "Hello, is anyone out there?",
    "timestamp": "2026-03-24T14:01:32"
  },
  ...
]
```

Your JSON structure doesn't have to match this example exactly. Design it however makes sense to you. The point is that you can `cat bbs.json` and see exactly what's stored. That's the beauty (and the limitation) of file-based storage.

Notice that search requires loading the entire file and scanning every message. Keep that in mind for Part B.

Tip: Use Python's built-in `json` and `datetime` modules. Your coding agent can help with the file I/O, but focus on understanding the data structure you're designing.

Part B: SQLite Upgrade

Now build a second version called `bbs_db.py` that stores data in a SQLite database using **SQLAlchemy with raw SQL** (not the ORM).

Database schema

Design and create two tables:

- **users** - at minimum: an integer primary key and a unique username
- **posts** - at minimum: an integer primary key, a user ID that references the users table, the message text, and a timestamp

Your program should create these tables automatically if they don't exist (i.e., on first run).

Required commands

Same as Part A:

```
python bbs_db.py post <username> <message>    # Post a message
python bbs_db.py read                          # Read all messages
python bbs_db.py users                        # List all users
python bbs_db.py search <keyword>             # Search posts by keyword
```

The search command

In your JSON version, search means loading the entire file and looping through every message. With SQL:

```
SELECT u.username, p.message, p.timestamp
FROM posts p JOIN users u ON p.user_id = u.id
WHERE p.message LIKE :keyword
```

One query. The database does the work.

Technical requirements

- Use `sqlalchemy` with `create_engine("sqlite:///bbs.db")`
- Use `text()` for all queries. Write real SQL, not ORM methods.
- Use parameterized queries (`:param` syntax). Never f-string your SQL.
- Create a `db.py` module that exports the engine and an `init_db()` function

Never do this:

```
f"SELECT * FROM posts WHERE message LIKE '%{keyword}%'"
```

This is a **SQL injection vulnerability**. Always use parameterized queries. Your coding agent knows this, but if it generates string-interpolated SQL, that's a sign you should double-check its output.

Part C: Migration

Write a script called `migrate.py` that reads `bbs.json` and populates the SQLite database.

```
python migrate.py                                # Reads bbs.json, writes to bbs.db
```

This is where the two halves of the assignment connect. Your flat JSON has usernames as strings on every post. Your database has a normalized `users` table with unique rows. The migration has to bridge that gap: extract the distinct users, create rows for them, then insert the posts with the correct user IDs. Timestamps from the JSON file should be preserved exactly.

After a successful migration, `bbs_db.py read` should produce the same output as `bbs.py read` for the same data. Verify this yourself before submitting.

Think about edge cases. What if the same username appears in many posts? What if `bbs.db` already has data in it when you run the migration? Decide on a behavior and document it in your README.

README Requirements

Include a `README.md` that covers:

1. How to run your program (any setup steps, dependencies)

2. A short comparison (a paragraph or two) of how search works in your JSON version vs. your SQL version. What's different about the approach? What would happen if your BBS had a million posts?
3. What your `migrate.py` does if `bbs.db` already exists (does it wipe it? skip duplicates? error out?) and why you chose that behavior
4. If you've added features for silver/gold: what you added and why

What to Submit

1. Create a branch: `git checkout -b bbs-<your-github-username>`
2. Put your code in a directory: `assignments/bbs/<your-github-username>/`
3. Push and open a PR titled "BBS - <your-name>"

Tiers

Bronze

Parts A, B, and C working as described. JSON version stores and reads posts and supports search. SQLite version has `users` and `posts` tables, supports all four commands (`post` , `read` , `users` , `search`). Migration script correctly moves data from JSON to SQLite. README covers all required sections including the search comparison and migration behavior.

Silver

Everything in Bronze, plus a meaningful feature that extends the BBS. Some ideas (pick one or invent your own):

- **Topics/boards** - posts belong to named boards (`python bbs_db.py post alice general "Hello!"`), with a command to list boards and read a specific one
- **Threads** - reply to a specific post by ID, display replies indented under the parent
- **User profiles** - a `profile` command that shows a user's join date, post count, and a short bio they can set

Include a brief explanation in your README of what you added and the schema changes it required.

Gold

Everything in Silver, plus something that makes your BBS genuinely impressive. Build something you'd want to show off. Some ideas to get you started:

- An ASCII art welcome screen and colored terminal output (look into ANSI escape codes or the `rich` library)
- Private messages between users
- An interactive mode: instead of one-shot commands, drop into a live session with a prompt (`bbs>`) that keeps you "logged in"
- A leaderboard, post reactions/upvotes, or a "trending" algorithm
- Import/export: dump the database to JSON, or load a JSON file into the database

Surprise us. The best Gold submissions are the ones where we can tell you had fun building them.

A Note on Coding Agents

You're encouraged to use your coding agent throughout this assignment. That said: an agent can probably produce working code for most of this quickly. That's fine, but it means the code is the floor, not the ceiling.

The real learning happens when you *direct* the agent toward something harder, debug the stuff it gets wrong, and make design decisions it can't make for you (like how to structure your threads table, or what "trending" means for your BBS). The README sections asking you to compare approaches and justify decisions are where you demonstrate understanding. Use the agent as a power tool, not an autopilot.

Grading

Bronze	Satisfactory: you demonstrated the core skills
Silver	Good: you extended the design thoughtfully
Gold	Excellent: you built something impressive